

Resit Part 2 - Lectures 3 to 4

Instructions:

- The assignments are individual. You may consult and discuss with your colleagues, but you need to provide independent answers.
- You may use Matlab/Python/Julia to solve the assignment. Many questions require you clearly to do so.
- Provide detailed answers, describing the steps you followed.
- Provide clear reports, typed, preferably using LaTeX.
- Submit the reports digitally on Brightspace as indicated on the lectures.
- Include your code, for reproducibility of your results, in your Brightspace submission on a zip file, or through a link on your report. Upload your report on Brightspace in a separate PDF file.
- The code will *only* be used to verify reproducibility in case of doubts. The grading will be performed based on the results described in the report.

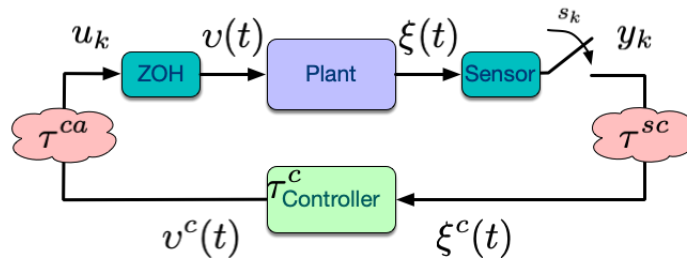


Figure 1: Schematic of a NCS including delays.

Consider a sample-data networked control system (NCS), as depicted in Figure 1. Assume that the plant dynamics are linear time-invariant given by:

$$\dot{\xi}(t) = A\xi(t) + Bv(t),$$

where the control signal v is a piece-wise constant signal resulting from the application of a controller in sampled-and-hold fashion, i.e. $v(t) = u_k, t \in [s_k, s_{k+1})$.

The system matrices are given by:

$$A = \begin{bmatrix} 0.5 - c & 0 \\ 0.2 + a - b & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

where a is given by the first digit, b by the third digit, and c by the last digit of your student ID number.

We consider again the case when the system is affected by a **constant** small delay $\tau \in [0, h)$, and it is controlled with the *static* controller $\bar{K}$ designed in Q1 of Part 1.

Question 4: (4p)

1. (2p) Employ the Jordan form approach to construct a polytopic over-approximation to study the stability of the system affected by delays in the interval $h \in [0, 1]$. Describe mathematically the approach.
2. (2p) Provide a plot illustrating the ranges of $h \in [0, 1]$ in which the polytopic approximation verifies stability of the system and comment on the differences with the result of question 2.1.

We consider again a different approach to deal with delays $\tau \in [0, h)$. If the delay experienced by an input upon arrival at the ZOH is $\tau \leq h/2$ the input is applied as usual, while if the input arrives with a delay $\tau \geq h/2$ the input is ignored and the previous input is held.

Let us consider now the case when delays are modelled stochastically as follows:

- The sequence of delays is initialized with equal probability on either delay.
- The probability of τ_2 following τ_1 is $p_1 = 0.8$
- The probability of τ_1 following τ_1 is $p_2 = 0.1$
- The probability of τ_1 following τ_2 is $q_1 = 0.5$
- The probability of τ_2 following τ_2 is $q_2 = 0.1$
- The probability of τ_1 following τ_3 is $r_1 = 0.2$
- The probability of τ_2 following τ_3 is $r_2 = 0.2$

Question 5: (6p)

1. (2p) Describe with a Markovian Jump Linear System the resulting system.

2. (1p) Derive the LMIs required to check Mean Square Stability (MSS) of this system.
3. (1p) Provide a plot depicting for your system the values of h resulting in a stable system.
4. (2p) Fix $r_1 = r_2 = r$. Provide a new plot (now 2D) illustrating for which values of h and r the resulting system is MSS. Discuss and interpret your results.

Question 6: (3p)

1. (3p) Consider your system being controlled by the static controller you designed for your continuous-time system $\bar{K}$. Design a quadratic event-triggered condition

$$\phi(\xi(t), \xi(s_k)) \leq 0$$

for the system to guarantee global exponential stability of the closed-loop. Simulate the resulting closed loop for various values of the σ parameter controlling the guaranteed performance of the closed-loop. Simulating for multiple initial conditions for each value of σ , compare the average amount of communications/sampling produced by each of the systems (different σ values) on a predefined fixed time-length of the simulations.

Hint: To simulate event-triggered control in continuous-time, you need to detect the triggering even while solving the ODE. The naïve way is to simulate at a very fine discrete-time rate, but this ends up being imprecise and computationally inefficient. ODE solvers in Python, Matlab, and Simulink, for example, have event-checking mechanisms that you can set up to, e.g., stop simulation at an state-dependent event. You can use this to stop simulation, update the control action, and restart the simulation in a loop. Here are some pointers for event detection:

- **Matlab:** any ODE solver, e.g., `ode45`, has an optional `options` input; look up `odeset` for implementation details (e.g., type `doc odeset` in the Matlab prompt and look for the “events” Section.)
- **Python:** similar to Matlab, `scipy` offers event-handling through an optional `events` argument. See the documentation for implementation details.
- **Simulink:** The Triggered subsystem is the recommended block to implement an event-based update of a signal.

If you are familiar with hybrid systems, particularly the jump-flow formalism, you may also consider the Matlab/Simulink toolbox HyEQ from UCSC.